

Supplementary Materials for “Improved Methods for k -core Community Search”

Ian Chen¹, Haotian Yi¹, Arun Sharma², George Chacko¹, and Tandy Warnow¹

¹ University of Illinois Urbana-Champaign, Urbana IL 61801, USA,
`{ianchen3,yi54,chackoge,warnow}@illinois.edu`,
² `arun@ladybugmem.ai`

1 Additional Figures

We provide figures that were omitted from the paper due to space limitations.

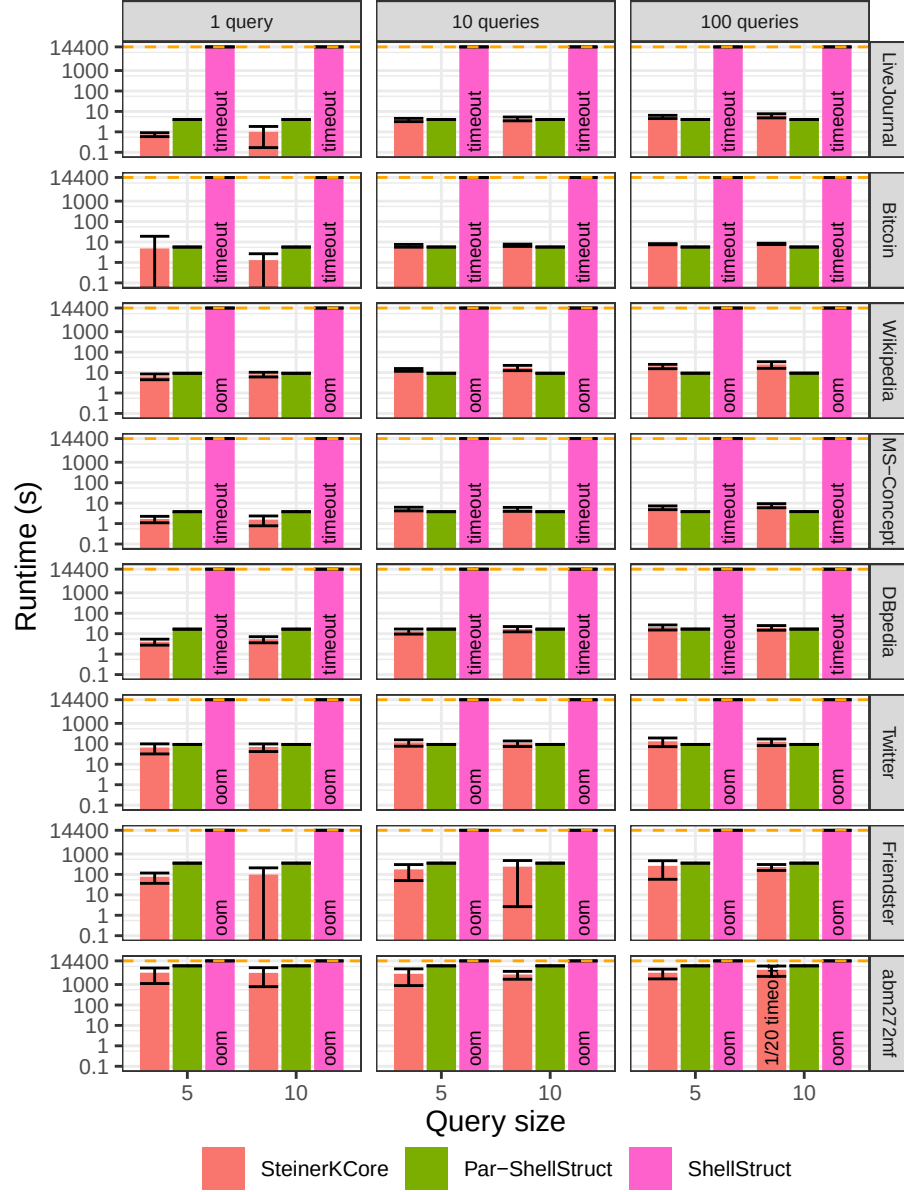


Fig. 1. Experiment 2: Runtimes of four community search methods (CSK, SteinerKCore, Par-ShellStruct, and ShellStruct) on testing networks given 16 cores: query sizes 5 and 10. Runtimes do not include the time for computing the core decomposition, as all use the same technique. Each row indicates a network, sorted by number of vertices (from $\sim 5M$ to $\sim 273M$). Each bar shows average (error bars are std dev) for 20 batches of queries, each with 1–100 queries with 5 or 10 vertices per query. CSK only takes as input single-vertex queries and so is not shown. Failures are out-of-memory (oom) or timeout.

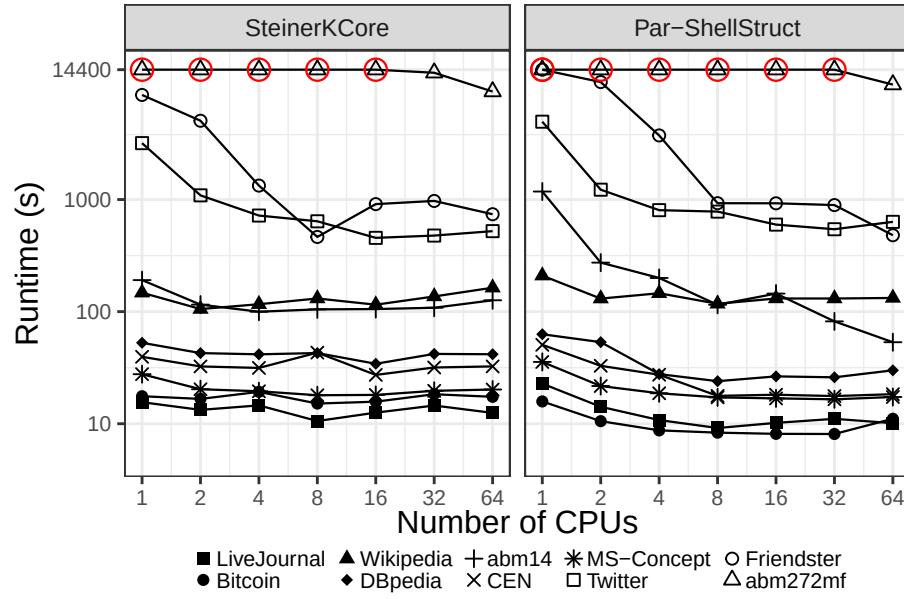


Fig.2. Experiment 3a: strong-scaling (all networks). Each facet shows the runtime of SteinerKCore and Par-ShellStruct, our community search methods. For 1, 2, 4, 8, 16, 32, 64 CPUs, we plot the mean runtime among 5 batches of 32 5-vertex queries. Neither method completes for abm272mf below 32 CPUs. Timeouts, indicated in red circles, occurred for SteinerKCore below 16 CPUs on abm272mf, and Par-ShellStruct below 32 CPUs for abm272mf and at 1 CPU on Friendster.